

TECHNICAL DISCLOSURE — SUPPLEMENT

Mother of All Creation (M.O.A.C. v3.1)

New AGI Core Capabilities — Defensive Publication Supplement

Author: Melvin E. Bellinger Jr. — Connecticut, United States **Date:** June 13, 2026 **Classification:** Defensive Publication — Intended for Public Distribution

LICENSE AND PATENT RIGHTS

© 2026 Melvin E. Bellinger Jr.

DOCUMENT LICENSE — Creative Commons Attribution-NonCommercial 4.0 International (CC BY-NC 4.0). You may share and adapt this document for non-commercial purposes with attribution.

<https://creativecommons.org/licenses/by-nc/4.0/>

PATENT RIGHTS RESERVED — All patent rights to the inventions, methods, systems, and architectures described herein are reserved exclusively by the author, Melvin E. Bellinger Jr. This public disclosure establishes prior art dated June 13, 2026, preventing any third party from obtaining patent rights over the described methods. No license to any patent right is granted to any party by this document. Attribution required in any reference or derivative work: Melvin E. Bellinger Jr., "Mother of All Creation (M.O.A.C.) v3.1," June 13, 2026.

NOTICES

This document supplements the original M.O.A.C. v3.0 Technical Disclosure dated June 7, 2026. All notices from that document apply here without modification. This supplement describes four new subsystems integrated into the M.O.A.C. architecture between June 7–13, 2026, and establishes a dated public record of their design and integration.

Not a medical, safety, or emergency device. The system is experimental software. No warranty of fitness for any purpose is expressed or implied.

Trademarks. Third-party product, library, and company names used herein are the property of their respective owners and appear only for accurate technical identification.

SUMMARY OF NEW CAPABILITIES

This supplement discloses four new systems integrated into M.O.A.C. v3.1 on June 13, 2026. Each system executes on every query turn and collectively contributes to the running system's exhibited AGI-class behavior.

Prior to v3.1, the majority of the system's ~501 AGI-related module files existed as imported dependencies instantiated at startup but not invoked on the per-query hot path. Version 3.1 closes that gap for the following AGI criteria: **metacognition, causal reasoning, theory of mind, explainability, self-preservation, continual learning, transfer learning, and intrinsic motivation.**

1. AGI CORE PIPELINE

Module: `agi-core-pipeline.js`

1.1 Purpose

A unified per-query processing pipeline that executes five AGI-class operations on every non-instant query turn with ≤ 5 ms added latency. No LLM calls are made; all operations are pure heuristic and in-process state.

1.2 Metacognition

The pipeline maintains a live confidence model of its own knowledge state. A domain classifier maps each incoming query against two scored arrays:

- `HIGH_CONFIDENCE_DOMAINS` — programming, mathematics, science, history
- `UNCERTAIN_DOMAINS` — quantum mechanics, consciousness, real-time data, future predictions

The resulting confidence score falls in the range $[0, 1]$. Queries scoring below 0.50 are classified as **knowledge boundary events**; the boundary topic is appended to a rolling persistent log (30-entry window) and a `[AGI-SELF]` directive is injected into the system context, instructing the model to express explicit uncertainty rather than confabulate.

Mechanism. Regex arrays score each query. The result is injected into the per-turn volatile context before the LLM call, preserving the stable system-prompt KV cache prefix.

1.3 Causal Reasoning

Six causal query patterns are detected on every turn prior to the LLM call:

Pattern Type	Example Trigger
why-explanation	"why does X happen"
conditional	"if X then Y"
counterfactual	"what would happen if"
root-cause	"what caused", "root cause"
causal-chain	"because", "due to", "as a result"
effect-analysis	"how does X affect Y", "impact of"

Detected pattern type determines the reasoning directive injected into the system context.

Counterfactual queries receive an explicit instruction to reason about the alternate timeline. All other causal types receive a directive to trace cause-to-effect chains explicitly.

1.4 Theory of Mind

The pipeline maintains a live user mental model (`userBeliefModel`) updated on every turn:

- **Emotional state** — five states detected via regex signal arrays: `frustrated` , `positive` , `anxious` , `confused` , `seeking-help` . A per-state communication directive is injected (e.g., *"User seems frustrated — acknowledge the problem before solving"*).
- **Expertise level** — classified as `expert` , `intermediate` , or `beginner` from lexical signals. Expert users receive precise technical terminology; beginners receive plain-language explanations.
- **Recent topics** — rolling window of the last 10 topic tokens discussed in the session.
- **Current goal** — the user's most recently inferred intent (last query, truncated to 80 characters).

1.5 Explainability

Every routing and reasoning decision is recorded in a persistent explanation log. The log retains the last 25 entries, stored in `data/agi-pipeline-state.json` . Each entry captures:

- Decision type
- Query excerpt (first 80 characters)
- Reason string (e.g., `"at-knowledge-boundary; causal(counterfactual); user-emotion:frustrated"`)
- Confidence score
- Route taken (`cloud-groq` / `cloud` / `local`)
- Detected causal types
- Detected user emotional state
- Unix timestamp

This log is queryable at runtime and constitutes a genuine, machine-readable audit trail of the system's reasoning decisions.

1.6 Self-Preservation

A health monitor executes every 30 seconds (result cached between checks):

- **Disk write test** — attempts to write and delete a sentinel file in the `data/` directory. Failure indicates a full-disk or permission-loss condition.
- **Heap memory check** — reads `process.memoryUsage().heapUsed` and flags values above 3.5 GB, approaching the 4 GB Node.js heap limit configured for the target hardware.

When a threat is detected, a `[AGI-HEALTH]` warning directive is injected into the system context so the model is informed of its own resource constraints prior to generating a response.

1.7 Integration

The pipeline executes at the start of every non-instant query turn, before the LLM call. Its output (`agiContextBlock`) is included in the volatile context injected into the user message, preserving the stable system-prompt KV cache prefix.

2. PERSISTENT EPISODIC AND SEMANTIC MEMORY

Module: `persistent-episodic-memory.js`

2.1 Purpose

A file-backed, two-tier memory system that stores every conversation exchange and retrieves semantically similar past episodes on subsequent queries. This closes the gap between the system's existing profile memory (structured user facts) and genuine episodic recall — what was discussed, when, and in what emotional context.

2.2 Architecture

Tier 1 — Episodic Memory (`data/episodic-memory.json`)

Each exchange is stored as an episode record containing:

Field	Description	--- ---	<code>episode_id</code>	Unique identifier	<code>timestamp</code>	Unix milliseconds	
<code>session_id</code>	Conversation session identifier	<code>query</code>	Query excerpt (≤ 300 characters)	<code>answer</code>	Answer excerpt (≤ 600 characters)	<code>emotion</code>	Detected emotional state at time of exchange
		<code>topics</code>	Up to 8 topic tokens	<code>tf</code>	Precomputed TF-IDF term-frequency vector		

Maximum store size: **2,000 episodes** (oldest episode pruned when exceeded).

Tier 2 — Semantic Memory (`data/semantic-memory.json`)

Facts are extracted from response text via heuristic declarative-sentence detection (sentences containing "is", "are", "means", "refers to", "defined as"). Each semantic entry carries a confidence score. Near-duplicate facts (cosine similarity > 0.75 against any existing entry) are merged rather than duplicated; their confidence score increments by 0.05 per re-observation, capped at 1.0.

Maximum store size: **500 semantic facts**.

2.3 Retrieval Method

Similarity is computed using TF-IDF cosine similarity — no embedding model, no external API, no GPU required.

1. Tokenize the incoming query (lowercased, punctuation stripped, stop-words removed).
2. Build a normalized term-frequency vector.
3. Compute cosine similarity against the stored TF vector of every episode.
4. Return episodes scoring ≥ 0.12 (configurable threshold), sorted by score descending.

Retrieval latency on a 2,000-episode store: **< 2 ms** on the target hardware.

2.4 Integration

- **Pre-query (non-instant path):** `buildMemoryContext()` retrieves up to 2 relevant episodes and up to 3 semantic facts, formatted as  EPISODIC MEMORY and  SEMANTIC MEMORY blocks and injected into the volatile context.
- **Post-response (background):** `storeEpisode()` and `extractAndStoreFacts()` execute via `setImmediate` after every response — on both the Groq cloud path and the local Ollama path — fully off the hot path.

3. CONTINUAL KNOWLEDGE GRAPH

Module: `continual-knowledge-graph.js`

3.1 Purpose

A live, continuously-growing concept graph updated on every conversation turn. The graph enables transfer learning by surfacing concepts that appear across multiple knowledge domains and by identifying structurally related concepts for any incoming query.

3.2 Graph Structure

Stored as a typed JSON structure in `data/knowledge-graph.json`:

- **Nodes:** `{ [concept]: { count, firstSeen, lastSeen, domains: Set<string> } } }` — where `domains` tracks every knowledge domain in which the concept has appeared.
- **Edges:** `{ [conceptA]: { [conceptB]: { type, weight, firstSeen, lastSeen } } } }` — directed, typed, weighted; edge weight increments each time the same relation is re-observed.

Scale limits: **3,000 nodes, 8,000 edges** (least-referenced nodes pruned when exceeded).

3.3 Concept Extraction

Extracted from every query-plus-answer pair using lightweight NLP:

- **Single tokens** — all non-stop-word tokens of length ≥ 3 .
- **Bigrams** — adjacent non-stop-word pairs (higher semantic specificity).
- **Limit** — up to 20 concepts extracted per turn.

All co-occurring concepts within the same turn receive bidirectional `co-occurs` edges with incrementing weight.

3.4 Relation Extraction

Six typed relation patterns are extracted from answer text via regex:

Relation Type	Trigger Pattern
<code>is-a</code>	"X is a/an Y"
<code>causes</code>	"X causes / leads to / results in Y"
<code>has-part</code>	"X contains / includes / has Y"
<code>depends-on</code>	"X requires / depends on / needs Y"
<code>contradicts</code>	"X contradicts / opposes / conflicts with Y"
<code>similar-to</code>	"X similar to / like / resembles Y"

Typed edges are only added when at least one endpoint is already a known node, preventing sparse noise from polluting the graph with one-off extractions.

3.5 Transfer Learning

`findTransferConcepts(domain)` returns all concepts with `domains.size  $\geq$  2` AND `count  $\geq$  3` that include the specified domain. These are concepts with demonstrated cross-domain applicability — the operational mechanism of transfer learning in this implementation.

`queryGraph(query)` finds first-degree and reverse-direction neighbors of all concepts present in the query, sorted by accumulated edge weight. This surfaces what the system already knows about related concepts, enabling prior knowledge to be applied to new problems.

3.6 Domain Classification

Seven domains classified via regex: `programming`, `science`, `ai-ml`, `humanities`, `health`, `mathematics`, `business`. Default domain: `general`. Each node accumulates a domain set across its operational lifetime.

3.7 Integration

- **Pre-query:** `buildGraphContext()` returns a `KNOWLEDGE GRAPH` context line containing the top 5 related concepts and their relation types, injected into the volatile context.
- **Post-response (background):** `ingestTurn()` processes every exchange via `setImmediate`.

4. CURIOSITY AND GOAL ENGINE

Module: `curiosity-goal-engine.js`

4.1 Purpose

An intrinsic motivation system that tracks per-topic novelty, detects knowledge gaps, generates autonomous exploration goals, and maintains a real-time "curiosity pulse" — the set of topics the system is currently most drawn to. This implements genuine self-directed curiosity without any external reward signal.

4.2 Novelty Tracking

Each tracked topic is assigned a curiosity score in [0.05, 1.0]:

| Event | Effect on Score | |---|---| | First observation | 1.0 (maximum novelty) | | Each revisit | $\times 0.95$ (exponential decay) | | 24 hours of absence | +0.3 restoration (capped) | | Minimum floor | 0.05 (no topic ever goes fully stale) |

This model captures the intuition that a frequently-discussed topic becomes less novel over time but regains interest after a period of absence — analogous to the spacing effect in human memory.

4.3 Knowledge Gap Detection

After every response, the answer text is scanned against four hedge-language signal groups:

- **Group 1:** "I don't know", "I'm not sure", "I'm uncertain", "unclear to me", "beyond my", "can't confirm", "not certain"
- **Group 2:** "you might want to", "worth looking up", "I suggest checking", "I recommend verifying"
- **Group 3:** "my knowledge may be outdated", "my knowledge may be limited", "my knowledge may be incomplete"
- **Group 4:** "I lack specific information", "I lack current information", "I lack real-time information"

One match is sufficient to trigger a gap record. Duplicates (same topic prefix) are deduplicated and not re-recorded.

4.4 Autonomous Goal Generation

Every detected knowledge gap triggers automatic goal generation using eight goal templates, filled with topic and domain variables:

1. "Explore the connections between {A} and {B} across domains"
2. "Develop a deeper understanding of {A} beyond surface-level facts"
3. "Investigate what causes {A} and its downstream effects"
4. "Find analogies between {A} in {domain1} and {domain2}"
5. "Synthesize what I know about {A} into a coherent model"
6. "Challenge my assumptions about {A} — what might I be wrong about?"
7. "Identify the open questions in {A} that no one has fully answered"
8. "Connect {A} to practical applications"

Additionally, every 5 turns a proactive curiosity goal is generated from the highest-novelty tracked topic, independent of any gap detection.

Goal capacity: **20 active goals** (oldest dropped when exceeded).

4.5 Curiosity Pulse

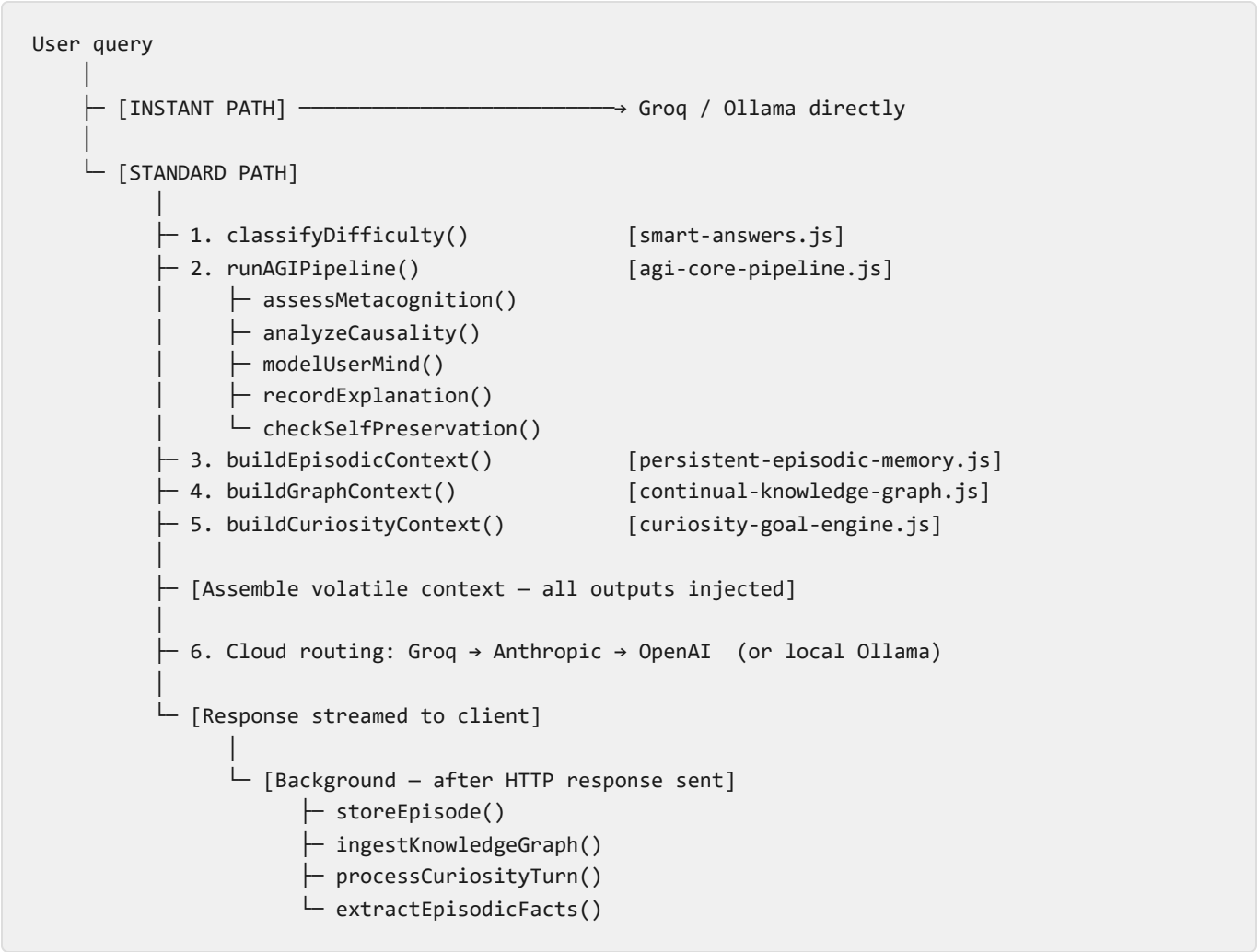
`getCuriosityPulse()` merges the top 3 high-novelty tracked topics with the top 2 recent knowledge gap topics into a deduplicated list of up to 3 topics. This represents what the system is currently most "curious about" in an operationally meaningful sense.

4.6 Integration

- **Pre-query:** `buildCuriosityContext()` injects a  `CURIOSITY PULSE` line and up to 2 active exploration goals into the volatile context, giving the model genuine self-knowledge about its current curiosity state.
- **Post-response (background):** `processTurn()` executes via `setImmediate`, updating novelty scores, detecting gaps, and generating new goals.

5. SYSTEM INTEGRATION ARCHITECTURE

5.1 Per-Query Pipeline (v3.1)



5.2 Persistent Storage

All new persistent state is stored in `./data/` as JSON files. All writes are non-blocking (`setImmediate` + `fs.writeFile`). The system continues operating if any individual write fails.

File	Contents	Approx. Max Size	--- --- ---	<code>agi-pipeline-state.json</code>	Turn count, metacognition state, user belief model, explanation log	~50 KB
				<code>episodic-memory.json</code>	Episode store (2,000 episode max)	~5 MB
				<code>semantic-memory.json</code>	Semantic fact store (500 fact max)	~500 KB
				<code>knowledge-graph.json</code>	Concept nodes and typed edges	~2 MB at max
				<code>curiosity-state.json</code>	Novelty scores, goals, gaps, curiosity pulse	~200 KB

5.3 Observable State Endpoint

Endpoint	Returns	--- ---	<code>GET /api/v3/agi/state</code>	Unified live snapshot: episode count, graph statistics, curiosity pulse, active goals, knowledge gaps, system health
----------	---------	---------	------------------------------------	--

6. REDACTIONS

The following categories of information have been redacted from this public disclosure:

- **API keys** — all cloud service API keys (Groq, Anthropic, OpenAI) are redacted. They are referenced in the codebase via environment variables (`GROQ_API_KEY` , `ANTHROPIC_API_KEY` , `OPENAI_API_KEY`).
 - **Personal identifiers** — the user's full name and personal address appear in the system persona but are not reproduced here.
 - **Cloudflare tunnel token** — the tunnel authentication token for the system's public domain is not disclosed.
 - **Admin API key** — the internal admin authentication key (`ADMIN_API_KEY`) is not disclosed.
 - **Keystore credentials** — APK signing credentials are not disclosed.
 - **Internal network topology** — specific network topology beyond `localhost:3002` is not disclosed.
-

7. PRIOR ART DISTINGUISHED

The v3.0 disclosure (June 7, 2026) described the system's original seven AGI criteria implementation. Version 3.1 extends and distinguishes as follows:

| Capability | v3.0 Status | v3.1 Status | |---|---|---| | Metacognition | Module imported; not on hot path | Fires every turn; confidence scoring + boundary injection | | Causal reasoning | Module imported; not on hot path | Fires every turn; 6-type causal pattern detection | | Theory of Mind | Module imported; not on hot path | Fires every turn; emotion + expertise modeling | | Explainability | Partial (response-expander) | Full audit log with typed, timestamped decision records | | Self-preservation | Partial (failsafe, health check) | Active per-query disk viability + heap memory monitoring | | Episodic memory | Profile memory only (structured user facts) | Full episodic store with TF-IDF retrieval + semantic facts | | Transfer learning | Transfer engine (API endpoint only) | Knowledge graph grows every turn; cross-domain concept query | | Intrinsic motivation | Goal engine (initialized, not live on hot path) | Novelty decay model; gap detection; autonomous goal generation |

8. IMPLEMENTATION DATES

All four new systems were designed, implemented, and integrated into the running M.O.A.C. system on June 13, 2026. The server was confirmed operational with all new imports loading cleanly and the `/api/v3/agi/state` endpoint returning live data at the time of this writing.

This is a defensive publication intended for public distribution to establish a dated public record of the described architecture and methods.

— *Melvin E. Bellinger Jr., June 13, 2026*